

VaultProof

A confidential solvency oracle built on Flare Confidential Compute. Prove you hold off-chain assets, without revealing the account, the balance, or the API key that proved it.

Study and build reference · Flare Confidential Compute bounty · Prashant Thakur

Contents

- 1 The pitch in one page
- 2 The problem, with a worked example
- 3 Why FTSO and FDC cannot do this, and FCC can
- 4 System architecture
- 5 End-to-end sequence, step by step
- 6 The cryptography: attestation-bound key exchange
- 7 Data minimisation: tiers, nullifiers, expiry
- 8 Smart contracts
- 9 The FCE extension handler
- 10 Threat model and attack walkthroughs
- 11 Build plan and milestones
- 12 Demo script
- 13 Glossary and study questions

1. The pitch in one page

A borrower on Flare has 2,000 USDC on-chain and wants an 8,000 USDC loan. Every lending protocol says no, because on-chain that address looks poor. In reality the borrower holds about \$52,000 on Kraken. The information exists; it just cannot be moved on-chain without handing someone an exchange API key and a full balance history.

VaultProof moves the *conclusion* on-chain without moving the data. The user seals a read-only exchange API key to a public key that only a hardware enclave holds. Inside Flare Confidential Compute, the enclave decrypts the key, queries the exchange, prices the holdings using Flare's own FTSO feeds, and emits a single signed statement:

```
address 0x7a3f...c19d | solvency tier T3 ($50k-$250k) | expires 2026-08-07T14:22Z
nullifier 0x9e21... | measurement 0x4b7c... | signed by the attested enclave
```

A lending contract reads that record and approves undercollateralised credit. What never exists outside the enclave: the API key, the exchange name, the asset mix, the exact balance, the account identifier. The lender is not trusting VaultProof the company. The lender is trusting a code hash registered in `TeeExtensionRegistry`, which anyone can rebuild and verify.

Why this fits a confidential compute bounty

Take the TEE away and the product does not become less private, it becomes impossible. There is no version of this that works with a normal server, because a normal server means somebody's ops team can read live exchange credentials. That is the test a confidential compute submission has to pass, and most submissions fail it: they build something that would work fine without a TEE and then bolt privacy on as a feature.

The second thing that matters here is what the enclave signature is *for*. It is consumed by a smart contract that moves money. It is not a receipt shown in a dashboard. The chain, the TEE signing key, and the attestation registry are all load-bearing at once.

2. The problem, with a worked example

DeFi lending is overcollateralised because a contract cannot see anything except the chain it runs on. Deposit \$150 of ETH, borrow \$100 of stablecoin. That is capital-inefficient and useless to anyone whose wealth is somewhere else, which is most people.

Traditional finance solves this with credit checks: the lender sees your income, your assets, your history. The lender learns everything about you, and you trust the lender to protect it. Both properties are bad. DeFi has the opposite failure: the lender learns nothing, so the lender demands collateral.

Worked example

Fact	Value	Who currently knows it
On-chain wallet balance	2,000 USDC on Flare	Everybody
Kraken holdings	0.42 BTC, 3.1 ETH	Kraken, the user
USD value of holdings	~\$52,400	Kraken, the user
Desired loan	8,000 USDC	The user
What the lender needs	"is this borrower good for \$8k?"	Nobody, on-chain

The gap is exactly one boolean, and there is no safe pipe to carry it. The user cannot paste an API key into a dapp. The dapp cannot ask Kraken directly, because Kraken has no idea who owns wallet 0x7a3f. A screenshot proves nothing.

VaultProof builds the pipe. The enclave is the only party that ever holds both halves of the secret at once, and it is a party with no operator, no shell access, and a published code hash.

3. Why FTSO and FDC cannot do this, and FCC can

Flare's three enshrined protocols each bring outside data on-chain, and it is worth being precise about which gap each one fills, because this is the argument that positions the project.

Protocol	Brings on-chain	Trust mechanism	Can it see private data?
FTSO	Price feeds	Many independent providers, stake-weighted median	No. Everything it reads is public by definition.
FDC	External chain events and public Web2 API responses	Independent verifiers each fetch the same URL and vote	No, and structurally cannot. Verification requires many parties to fetch the same thing, so the thing must be public.
FCC	Results computed over private inputs	Hardware attestation of a whitelisted code hash	Yes. One party computes, and hardware vouches for what code ran.

That FDC row is the important one. FDC's security comes from replication: if ten verifiers independently fetch an API and agree, you can trust the answer. Replication is incompatible with secrecy. You cannot ask ten verifiers to check a balance behind one API key without giving ten verifiers the API key.

FCC inverts the trust model. Instead of trusting a quorum of parties, you trust a piece of silicon and a build you can reproduce yourself. That is what makes private inputs possible at all.

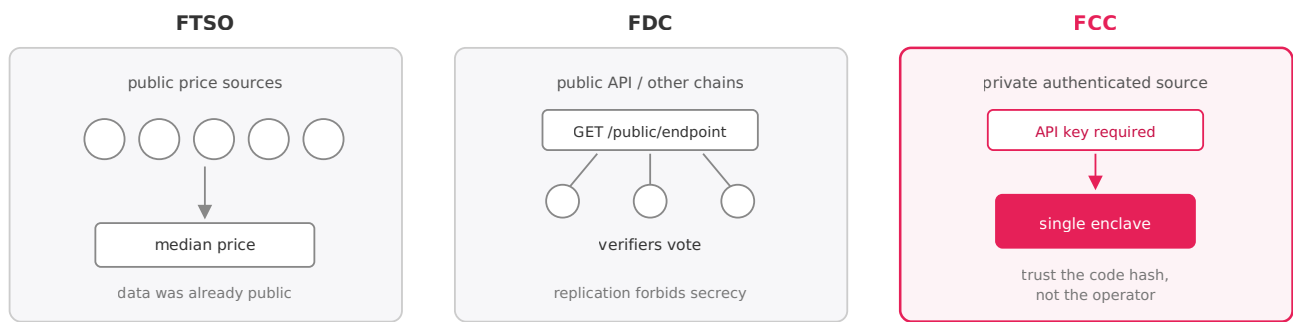


Figure 1. Each protocol closes a different gap. Only FCC can consume an input that must stay secret.

One-line positioning for the pitch deck: **FDC brings public data on-chain. FCC brings private conclusions on-chain. VaultProof is the first thing built in that second category.**

4. System architecture

Five components. The design rule that generates all of them: anything that touches a secret lives inside the enclave, and everything else is assumed hostile, including our own backend.

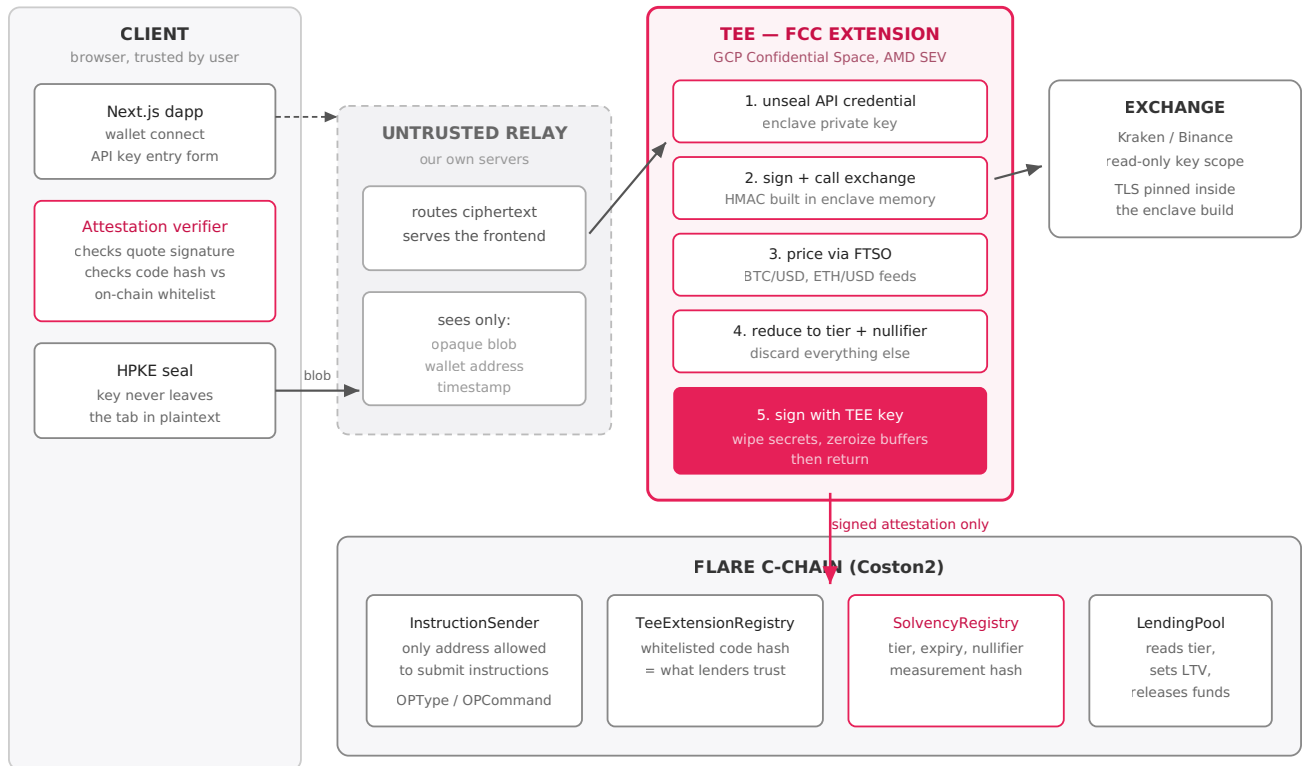


Figure 2. Architecture. The relay is drawn dashed because it is deliberately worthless to attack.

Component responsibilities

Component	Stack	Holds secrets?	Job
Dapp	Next.js 14, wagmi, viem, hpke-js	Briefly, in the tab	Verify attestation, seal credential, submit instruction, render result
Relay	Any host	No	Serve frontend, forward opaque payloads. Compromise gains nothing.
FCE extension	Go, from <code>fce-extension-scaffold</code>	Yes, all of them	Unseal, fetch, price, reduce, sign, wipe
Contracts	Solidity, Foundry	No	Route instructions, store attestations, enforce lending policy
Verifier CLI	Go or TS	No	Reproduce the build, confirm the on-chain code hash matches source

5. End-to-end sequence, step by step

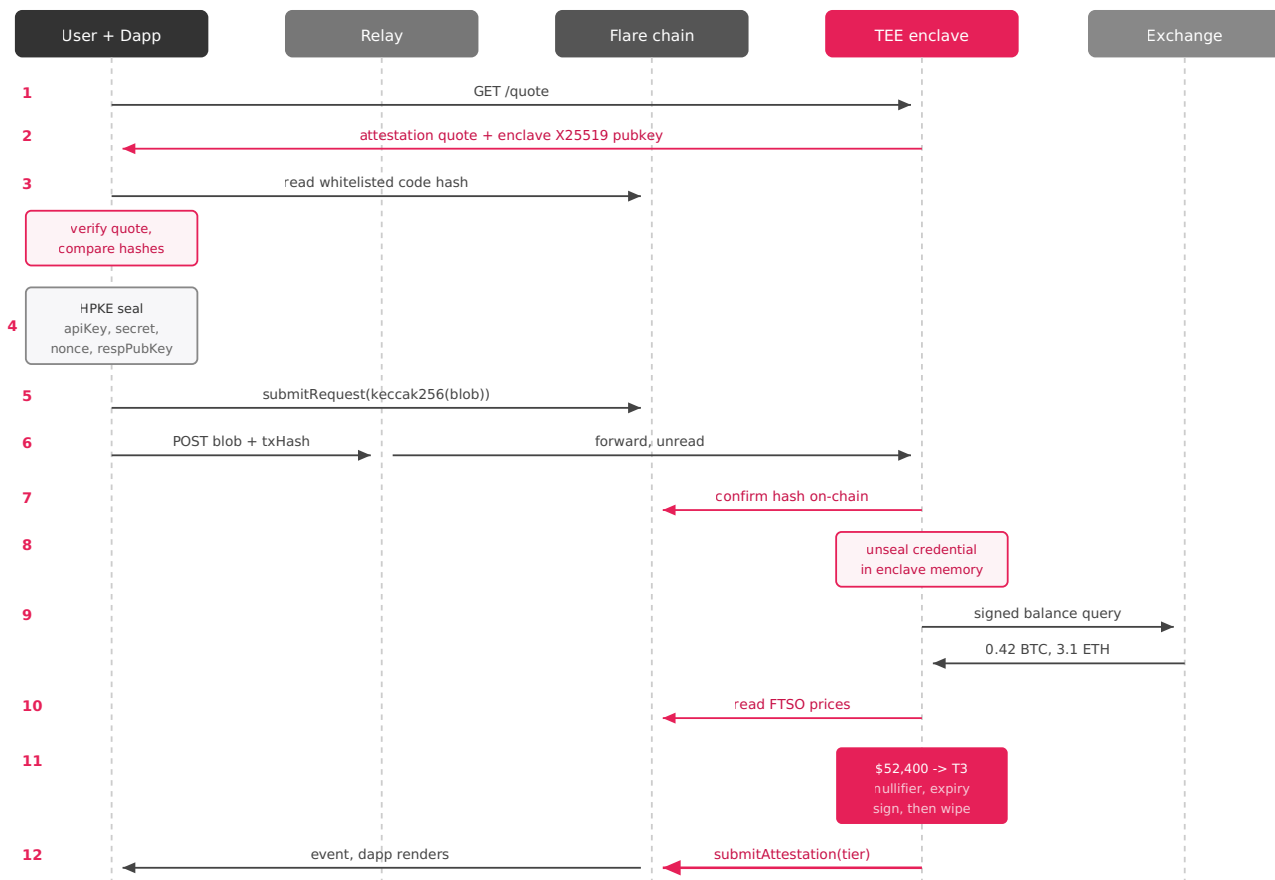


Figure 3. The full sequence. Steps 1-3 exist only so the user never has to trust us; skip them and the encryption is decorative.

Step 1-2: fetch the quote

The dapp asks the enclave for an attestation quote before doing anything else. The quote is produced by the Confidential Space environment and includes the measurement of the running image, plus a payload field we control. Into that payload we put the enclave's freshly generated X25519 public key.

```

GET https://fce.vaultproof.xyz/quote

{
  "measurement": "0x4b7c9e02a1...",
  "extensionVersion": "vaultproof-v0.3.1",
  "enclavePubKey": "0x2f7a1c...", // X25519, generated at boot, never persisted
  "quote": "eyJhbGciOiJSUzI1NiIs...", // signed by the platform
  "mode": 0 // 0 = real hardware attestation
}
  
```

Step 3: verify before trusting

The dapp does two checks, and both matter:

1. The quote signature chains to a genuine AMD SEV / Confidential Space root. This proves it is real hardware in a real confidential VM.
2. The measurement in the quote equals a code hash registered in `TeeExtensionRegistry` on Coston2. This proves the hardware is running *our published build* and not something an attacker swapped in.

Only after both pass does the dapp treat `enclavePubKey` as safe to encrypt to. This is the hinge of the whole system. A user who skips verification is back to trusting a server.

Step 4: seal the credential

```
// in the browser, hpke-js
const payload = {
  exchange: "kraken",
  apiKey: "vN5xL2...", // read-only scope, user-generated
  apiSecret: "aG9sZG...",
  wallet: "0x7a3f...c19d",
  nonce: "0x8812ffae...", // single use, checked on-chain
  respPubKey: "0x91bb...", // ephemeral, for sealing the reply back to us
  requestedAt: 1754489400
};

const sealed = await hpke.seal(enclavePubKey, JSON.stringify(payload));
// sealed = 0x04a1... (opaque, ~600 bytes)
```

Step 5-6: anchor, then deliver

The hash goes on-chain first, the ciphertext goes off-chain second. That ordering is deliberate. Putting the ciphertext itself in calldata would publish it permanently, so a future compromise of the enclave key would retroactively expose every credential ever submitted. Hashes leak nothing and still give a public, ordered, tamper-evident record that a request happened.

Step 7: bind request to chain

The enclave refuses to process a blob whose `keccak256` does not appear in a confirmed `RequestSubmitted` event, and refuses any nonce it has seen before. This stops the relay from silently replaying or fabricating requests, and it means every action the enclave takes has a paid-for on-chain footprint.

Step 8-9: unseal and fetch

The credential is decrypted into enclave memory. The HMAC signature that Kraken's private endpoints require is computed there too, so the secret is never even serialised. The request goes out over TLS with the hostname and certificate pinned in the build, which is what prevents a compromised host from redirecting the call to a fake exchange.

```
POST https://api.kraken.com/0/private/Balance
API-Key: vN5xL2...
API-Sign: HMAC-SHA512(base64secret, path + SHA256(nonce + postdata))

{ "result": { "XXBT": "0.4200000000", "XETH": "3.1000000000", "ZUSD": "180.00" } }
```

Step 10: price it with FTSO

Prices come from Flare's own oracle rather than from the exchange. If the enclave took the exchange's quoted valuation, a user running a compromised or self-hosted "exchange" could inflate their own worth. FTSO is decentralised, already on the chain the loan settles on, and free to read.

```
0.42 BTC x $118,400 = $49,728
3.1 ETH x $2,730 = $8,463
USD balance = $180
-----
total = $58,371 -> tier T3
```

Step 11-12: reduce, sign, forget

Everything specific is discarded here. What survives is a tier, a nullifier, an expiry, and a signature. The enclave zeroizes the credential buffers before returning, and the reply the user receives is sealed to the ephemeral key from step 4, so even the relay carrying it back learns nothing.

6. The cryptography: attestation-bound key exchange

The single most common mistake in TEE projects, and the thing a sharp judge will probe: encrypting to a key you were handed by an untrusted server is not encryption, it is a formality. If the relay can substitute its own public key, it decrypts everything and re-encrypts to the enclave, and nobody notices.

The fix is to make the key inseparable from the attestation. The enclave generates the keypair itself at boot and embeds the public half inside the signed quote. The signature is over the measurement *and* the payload together, so a substituted key breaks the signature.

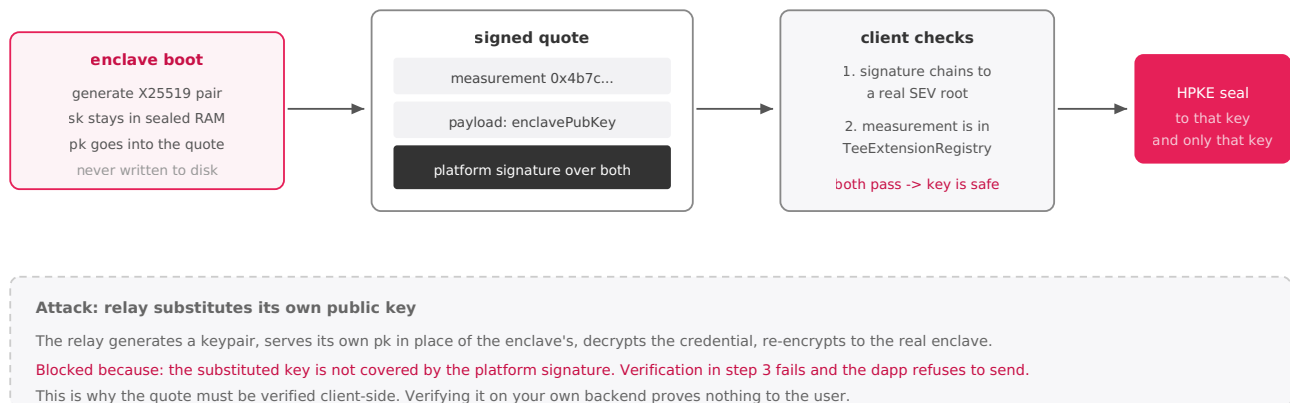


Figure 4. Binding the transport key to the attestation is what upgrades "encrypted" into "verifiably encrypted to attested code".

Choosing HPKE over hand-rolled crypto

HPKE (RFC 9180) gives a single-shot public-key encryption interface with X25519, HKDF-SHA256, and ChaCha20-Poly1305. Use `hpke-js` in the browser and `github.com/cloudflare/circl/hpke` in Go. Do not build this from raw NaCl primitives under hackathon time pressure; nonce reuse bugs are quiet and fatal.

Bind context so a sealed blob cannot be replayed against a different enclave version:

```
info = "vaultproof/v1|" + measurement + "|" + chainId
```

7. Data minimisation: tiers, nullifiers, expiry

An enclave that publishes "this wallet holds \$58,371.42" has protected the API key and leaked the user. Deciding what *not* to publish is as much of the design as the encryption is.

Tiers instead of amounts

Tier	Range	Suggested max loan	Bits leaked
T0	under \$1,000	none	~2.5
T1	\$1,000 - \$10,000	\$2,000	~2.5
T2	\$10,000 - \$50,000	\$8,000	~2.5
T3	\$50,000 - \$250,000	\$40,000	~2.5
T4	over \$250,000	\$150,000	~2.5

A tier is roughly two and a half bits of information. An exact balance is a fingerprint: unique, linkable across chains, and permanent once it is on a public ledger.

Nullifiers stop one account backing ten loans

Without this, a user attests the same Kraken account from five wallets and borrows five times against the same \$52k. The enclave derives a stable, unlinkable identifier from the exchange account:

```
nullifier = HMAC-SHA256(enclaveDerivedSecret, exchange || accountId)
```

It is stable, so the same account always yields the same nullifier. It is unlinkable, because the secret never leaves the enclave, so nobody can go from a nullifier back to an account. The registry rejects a second live attestation carrying a nullifier that is already bound to a different wallet.

Expiry

Balances move. An attestation is valid for 24 hours and the lending pool checks freshness at drawdown, not just at approval. Longer windows mean a user can attest, withdraw everything, and borrow against a fiction.

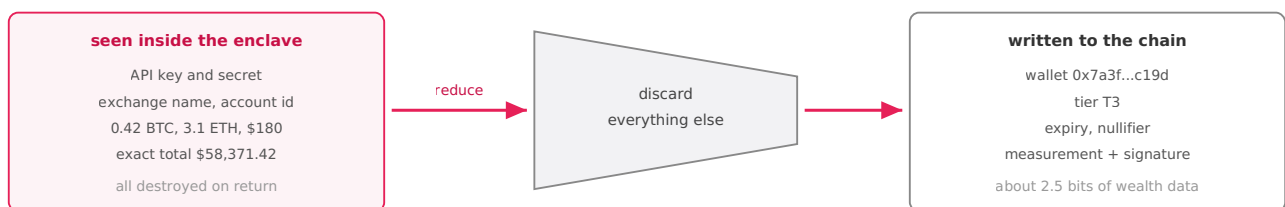


Figure 5. The reduction step is the privacy product. Encryption protects the pipe; minimisation protects the user from the output.

8. Smart contracts

Three contracts, kept deliberately small. Solidity 0.8.25, Foundry, deployed to Coston2.

SolvencyRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

contract SolvencyRegistry {
    struct Attestation {
        uint8 tier; // 0..4
        uint64 issuedAt;
        uint64 expiresAt;
        bytes32 nullifier; // stable per exchange account, unlinkable
        bytes32 measurement; // which enclave build produced this
    }

    address public immutable instructionSender; // only caller allowed
    mapping(address => Attestation) public attestations;
    mapping(bytes32 => address) public nullifierOwner;
    mapping(bytes32 => bool) public usedNonce;

    event Attested(address indexed wallet, uint8 tier, uint64 expiresAt, bytes32 measurement);
    event Revoked(address indexed wallet);

    error NotInstructionSender();
    error NullifierBound();
    error NonceReplayed();
    error StaleMeasurement();

    constructor(address _sender) { instructionSender = _sender; }

    modifier onlySender() {
        if (msg.sender != instructionSender) revert NotInstructionSender();
        _;
    }

    function submitAttestation(
        address wallet,
        uint8 tier,
        uint64 validForSeconds,
        bytes32 nullifier,
        bytes32 nonce,
        bytes32 measurement
    ) external onlySender {
        if (usedNonce[nonce]) revert NonceReplayed();
        if (!ITeeExtensionRegistry(TEE_REGISTRY).isWhitelisted(measurement)) revert StaleMeasurement();

        address bound = nullifierOwner[nullifier];
        if (bound != address(0) && bound != wallet) {
            // the same exchange account already backs a different wallet
            if (block.timestamp < attestations[bound].expiresAt) revert NullifierBound();
        }

        usedNonce[nonce] = true;
        nullifierOwner[nullifier] = wallet;
        attestations[wallet] = Attestation({
            tier: tier,
            issuedAt: uint64(block.timestamp),
            expiresAt: uint64(block.timestamp) + validForSeconds,
            nullifier: nullifier,
            measurement: measurement
        });

        emit Attested(wallet, tier, uint64(block.timestamp) + validForSeconds, measurement);
    }

    function tierOf(address wallet) public view returns (uint8) {
        Attestation memory a = attestations[wallet];
        if (block.timestamp >= a.expiresAt) return 0; // expired reads as no credit
        return a.tier;
    }
}
```

Two details worth defending in a demo. Expiry is enforced in the read path, not by a cleanup job, so a stale attestation cannot be used even if nobody prunes it. And `measurement` is checked against the registry at write time, so an old enclave build that gets de-whitelisted after a bug cannot keep issuing.

LendingPool.sol, the consuming side

```
uint256[5] public tierCap = [0, 2_000e6, 8_000e6, 40_000e6, 150_000e6];

function borrow(uint256 amount) external {
    uint8 tier = registry.tierOf(msg.sender);
    require(tier > 0, "no valid attestation");

    uint256 onChainCollateralValue = _collateralValue(msg.sender);
    uint256 cap = tierCap[tier] + onChainCollateralValue; // off-chain worth extends the cap
    require(borrowed[msg.sender] + amount <= cap, "over cap");

    borrowed[msg.sender] += amount;
    asset.transfer(msg.sender, amount);
}
```

Notice the attestation extends the borrowing cap rather than replacing collateral entirely. Undercollateralised does not mean uncollateralised, and saying so out loud makes the design look considered rather than naive.

InstructionSender wiring

`InstructionSender` is the only address permitted to submit instructions into the FCC flow, and routing uses `OPType` / `OPCommand` byte pairs that must match across the Solidity side, the Go router config, and the handler. Keep them in one shared constants file and generate both sides from it, because a silent mismatch here produces a request that vanishes with no error.

```
bytes32 constant OP_TYPE      = keccak256("VAULTPROOF");
bytes32 constant OP_SOLVENCY = keccak256("ATTEST_SOLVENCY");
```

9. The FCE extension handler

The extension is a Go service built from `fce-extension-scaffold`, exposing `POST /action`, following the four-step handler pattern, with signing done through the TEE signing port so the private key is never in your process memory.

```
func (h *Handler) HandleAction(w http.ResponseWriter, r *http.Request) {
    var req ActionRequest
    json.NewDecoder(r.Body).Decode(&req)

    // 1. decode and validate the instruction
    if req.OpType != OP_TYPE || req.OpCommand != OP_SOLVENCY {
        http.Error(w, "unknown op", 400); return
    }
    if err := h.chain.ConfirmRequestHash(req.TxHash, keccak256(req.Payload)); err != nil {
        http.Error(w, "unanchored request", 403); return
    }

    // 2. unseal the credential – first and only point where plaintext exists
    plain, err := h.hpke.Open(req.Payload, contextInfo(h.measurement))
    if err != nil { http.Error(w, "unseal failed", 400); return }
    defer zeroize(plain)

    var cred Credential
    json.Unmarshal(plain, &cred)
    defer zeroize([]byte(cred.APISecret))

    // 3. do the confidential work
    balances, err := exchange.FetchBalances(r.Context(), cred) // TLS pinned
    if err != nil { http.Error(w, "exchange call failed", 502); return }

    usd := h.ftso.ValueOf(balances) // price with Flare's own feeds
    tier := TierFor(usd)
    null := h.nullifier(cred.Exchange, balances.AccountID)

    // 4. sign via the TEE signing port and return
    att := Attestation{
        Wallet: cred.Wallet, Tier: tier, ValidFor: 86400,
        Nullifier: null, Nonce: cred.Nonce, Measurement: h.measurement,
    }
    sig, err := h.signer.Sign(att.Encode()) // key lives behind the signing port
    if err != nil { http.Error(w, "sign failed", 500); return }

    respond(w, sealToClient(cred.RespPubKey, att, sig))
}
```

Rules for everything under the enclave package

These are worth writing into a lint rule rather than a code review comment, because a single stray log line destroys the entire security claim and it will happen at 3am.

- No `fmt.Print`, `log.`, or any logger inside packages that touch credentials. Ban it in CI with a `grep`.
- Credentials live in function scope, are zeroized in a `defer`, and are never placed in a struct field that outlives the request.
- No credential-derived value in an error message. Return `"exchange call failed"`, never the upstream body.
- No panics with dynamic values, since a stack trace can carry a secret out.
- Reproducible builds: pin `SOURCE_DATE_EPOCH`, no timestamps or paths in the image, so a third party rebuilding your source gets a hash equal to the whitelisted one.

That last point is the one that turns "trust us" into "verify us". Publish the build command in the README and let a judge run it. If their hash matches what is registered in TeeExtensionRegistry, you have proven your claim on stage without saying a word.

10. Threat model and attack walkthroughs

Who can see what

Party	API key	Exact balance	Exchange used	Tier
The user	yes	yes	yes	yes
The enclave, during one request	yes	yes	yes	yes
VaultProof operators	no	no	no	yes, it is public
The relay host or cloud provider	no	no	no	yes
The lending protocol	no	no	no	yes
Anyone reading the chain	no	no	no	yes

Attack 1: the relay swaps the enclave public key

Attempt: our own compromised backend serves an attacker keypair, decrypts credentials in transit, forwards a re-encrypted copy so nothing looks broken.

Blocked by: the public key is inside the platform-signed quote. A substituted key invalidates the signature, and client-side verification fails before any credential is sealed. The failure is loud and happens on the user's machine, not ours.

Attack 2: one exchange account, ten wallets

Attempt: attest the same \$52k Kraken account from ten addresses, borrow ten times, walk away with \$80k.

Blocked by: the nullifier is derived from the exchange account id, so all ten attestations collide. The registry refuses to bind a live nullifier to a second wallet.

Residual risk: ten separate exchange accounts each holding \$52k is a real and expensive Sybil. Mitigation is the tier cap plus partial on-chain collateral, and the honest answer in Q&A is that this bounds the attack economically rather than eliminating it.

Attack 3: attest, then withdraw

Attempt: prove T3, immediately move the funds off Kraken, then borrow.

Blocked by: 24-hour expiry plus a freshness check at drawdown rather than at approval. A protocol wanting stronger guarantees can require re-attestation before each tranche.

Residual risk: the window is never zero. This is a credit-risk parameter, not a cryptographic one, and framing it that way is more credible than claiming it is solved.

Attack 4: a fake exchange endpoint

Attempt: point the enclave at a self-hosted server that reports 500 BTC, either by DNS poisoning or by submitting a custom endpoint field.

Blocked by: the exchange list is compiled into the build, hostnames and certificate pins included. There is no user-supplied URL field anywhere in the payload. Changing the allowed set changes the code hash, which is publicly visible.

Attack 5: the operator reads memory

Attempt: SSH into the host, attach a debugger, dump the process.

Blocked by: AMD SEV encrypts guest memory with a key the host cannot access, and Confidential Space does not give the project owner a shell into the workload. This is the property being bought from the hardware, and it is the one thing in the system that cannot be replicated in software.

Attack 6: replay a signed attestation

Attempt: capture a valid T4 attestation from the mempool and resubmit it later or for another wallet.

Blocked by: the wallet address is inside the signed payload, the nonce is single-use and marked on-chain, and expiry is enforced on read.

Attack 7: a malicious enclave build

Attempt: the team ships a new image that quietly exfiltrates credentials.

Blocked by: the new image has a different measurement. Until it is whitelisted it is rejected by the registry, and once whitelisted, anyone can rebuild from source and compare hashes. The trust anchor is the build, not the team.

Residual risk: whoever controls whitelisting is a governance dependency. Say so plainly instead of implying the system is trustless end to end.

Honest limitations

Three things that belong on a slide rather than buried in an appendix, because volunteering them reads as competence and getting caught hiding them does not.

- Attestation freshness. A quote proves the code that booted, not the code running at request time. Short-lived quotes and per-request re-fetch narrow this; nothing closes it fully.
- Side channels. AMD SEV has a published history of them. Adequate for a hackathon and for real money at moderate size; not a claim of perfection.
- Exchange trust. If Kraken lies about a balance, VaultProof faithfully attests a lie. The system proves what a named source said, not what is true.

11. Build plan and milestones

Ordered so every stage is demoable on its own. If time runs out at milestone 5 you still have a complete story on stage.

#	Milestone	Done when	Est.
1	Contracts on Coston2	SolvencyRegistry and LendingPool deployed, Foundry tests green including nullifier collision and expiry	4h
2	Extension skeleton	fce-extension-scaffold running in simulated mode, /action returns a hardcoded T2 signed by the TEE signing port, lands on-chain	5h
3	HPKE roundtrip	A CLI seals a payload to the enclave key, the enclave unseals it, the reply is sealed back and read. Unit tests both sides.	4h
4	Exchange integration	Real read-only Kraken key, live balance fetch, FTSO pricing, tier computed from real data	5h
5	Frontend	Wallet connect, quote fetch, client-side attestation verification, seal, submit, render attestation with explorer links	7h
6	Verification story	Reproducible build documented, verifier CLI that rebuilds and compares against the on-chain whitelist	3h
7	Real attestation	MODE=0 on actual Confidential Space, real quote verified by the client	4h
8	Polish	Demo rehearsed, seeded accounts, fallback recording, README with threat model up top	3h

Build in simulated mode for milestones 2 through 5 and flip to real hardware at 7. Debugging attestation and business logic simultaneously is how teams lose a night.

Repository layout

vaultproof/	
contracts/	Foundry: SolvencyRegistry, LendingPool, constants
extension/	Go: FCE extension from fce-extension-scaffold
handler/	/action, the 4-step pattern
secure/	credential handling, zeroize helpers, no logging allowed
exchange/	Kraken client, pinned TLS
ftso/	price reads
web/	Next.js dapp, hpke-js, attestation verifier
verifier/	CLI: rebuild image, compare hash to registry
docs/	this document, threat model, demo script

12. Demo script

Four minutes, one hard cut, one moment that lands. Practise the middle section until the timing is boring.

1. **Frame the gap, 30s.** Show a lending dapp refusing an 8,000 USDC loan to a wallet with 2,000 USDC. Say the borrower has \$52k on Kraken and no way to prove it without handing someone an API key.
2. **Show what you cannot do today, 20s.** One slide: FTSO does public prices, FDC does public APIs, and neither can touch anything behind a credential because verification by replication requires the data to be public.
3. **Verify before trusting, 40s.** Click "check enclave" in the dapp. Show the quote, the measurement, and the same hash sitting in TeeExtensionRegistry on the explorer. State clearly that the browser refuses to encrypt anything until those match.

4. **Run it, 60s.** Paste a real read-only Kraken key, submit, watch the pipeline stepper. Attestation appears: tier T3, expiry, nullifier, measurement.
5. **Borrow, 30s.** The same loan now goes through. 8,000 USDC lands in the wallet.
6. **The turn, 40s.** Open your own database and your own logs on screen. Grep for the API key. Nothing. Then say the line: *"I run this project and I cannot tell you which exchange that was, or how much they hold. Neither can my cloud provider. The lender trusted a code hash, and you can rebuild it yourself right now."*
7. **Roadmap, 20s.** Proof of income from bank APIs, proof of reserves for custodians, private KYC assertions. Same enclave, different reducer function.

13. Glossary and study questions

Term	Meaning
TEE	Trusted Execution Environment. Hardware-isolated compute where memory is encrypted against the host and hypervisor.
AMD SEV-SNP	The AMD technology behind GCP Confidential Space. Encrypts VM memory with a key the host cannot read.
Attestation quote	A hardware-signed statement of what code is running, plus a payload field the workload controls.
Measurement / code hash	Hash of the loaded image. The identity of the software, independent of who runs it.
TeeExtensionRegistry	Flare contract holding whitelisted code hashes. The on-chain source of truth for which builds are trusted.
InstructionSender	The only address permitted to submit instructions into the FCC execution flow.
OPType / OPCommand	bytes32 routing pair matched across Solidity, Go config, and the router.
MODE=0	Real hardware attestation, as opposed to simulated mode used during development.
HPKE	RFC 9180 hybrid public key encryption. Seal a message to a public key with one call.
Nullifier	A stable, unlinkable identifier derived from a secret input, used to prevent reuse without revealing the input.

Questions to test yourself before the pitch

1. A judge says "why not just use a ZK proof?" Answer covering live authenticated data, why the exchange would need to sign its response for ZK to work, and why TLS notary schemes are heavier than a TEE here.
2. Why must the attestation quote be verified in the browser rather than on your backend? What exactly breaks if you move that check server-side?
3. Why does the request hash go on-chain but the ciphertext does not? Name the specific future attack the ciphertext choice would enable.
4. Why price with FTSO instead of using the exchange's own USD valuation?
5. What does the nullifier prevent, and what does it deliberately not prevent?
6. Why is expiry checked in `tier0f` rather than by a cleanup transaction?
7. An auditor asks what happens if your team is compromised and pushes a malicious build. Walk the full chain of what stops it and where the residual governance risk sits.
8. Where does the trust actually terminate in this system? Name every party you still have to trust and why each is acceptable.